DATA ANALYTICS LAB SHEET 1

ANURAG PARMAR
ROLL NO : 8
SECTION : B
CUID : CU24250076
B.TECH CSE

Experiment No. 1

Real-World Data Collection, Cleaning and Preprocessing using Python

AIM

To collect a real-world dataset from a publicly available source and perform data preprocessing by handling missing values, duplicate records, inconsistent formats, categorical variables, and outliers using Python.

PURPOSE

In real-world analytics projects, data collected from multiple sources is often incomplete, inconsistent, and noisy. Before visualization or machine learning, it is essential to preprocess the data to improve quality and reliability.

DATASET

The Titanic Survival Dataset is used for this experiment.

NOTE

No manual dataset upload is required. The notebook automatically fetches the Titanic dataset from a public online dataset source when run in Google Colab.

1. Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler, MinMaxScaler

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

2. Collect and Load the Real-World Dataset

```
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv"

df = pd.read_csv(url)

print("Dataset loaded successfully.")
print("Dataset shape:", df.shape)
```

```
Dataset loaded successfully.
Dataset shape: (891, 15)
```

3. Inspect Dataset Structure

```
print("First five records:")
display(df.head())

print("\nDataset information:")
df.info()

print("\nStatistical description:")
display(df.describe(include="all"))
```

```
First five records:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

```
Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    object
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 6   fare         891 non-null    float64
 7   embarked     889 non-null    object
 8   class        891 non-null    object
 9   who          891 non-null    object
 10  adult_male   891 non-null    bool
 11  deck         203 non-null    object
 12  embark_town  889 non-null    object
 13  alive        891 non-null    object
 14  alone        891 non-null    bool
dtypes: bool(2), float64(2), int64(4), object(7)
memory usage: 92.4+ KB

Statistical description:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	emba
count	891.000000	891.000000	891	714.000000	891.000000	891.000000	891.000000	889	891	891	891	203	
unique	NaN	NaN	2	NaN	NaN	NaN	NaN	3	3	3	2	7	
top	NaN	NaN	male	NaN	NaN	NaN	NaN	S	Third	man	True	C	Sou
freq	NaN	NaN	577	NaN	NaN	NaN	NaN	644	491	537	537	59	
mean	0.383838	2.308642	NaN	29.699118	0.523008	0.381594	32.204208	NaN	NaN	NaN	NaN	NaN	
std	0.486592	0.836071	NaN	14.526497	1.102743	0.806057	49.693429	NaN	NaN	NaN	NaN	NaN	
min	0.000000	1.000000	NaN	0.420000	0.000000	0.000000	0.000000	NaN	NaN	NaN	NaN	NaN	
25%	0.000000	2.000000	NaN	20.125000	0.000000	0.000000	7.910400	NaN	NaN	NaN	NaN	NaN	
50%	0.000000	3.000000	NaN	28.000000	0.000000	0.000000	14.454200	NaN	NaN	NaN	NaN	NaN	
75%	1.000000	3.000000	NaN	38.000000	1.000000	0.000000	31.000000	NaN	NaN	NaN	NaN	NaN	
max	1.000000	3.000000	NaN	80.000000	8.000000	6.000000	512.329200	NaN	NaN	NaN	NaN	NaN	

4. Identify Missing Values

```
missing_values = df.isnull().sum()

print("Missing values in each column:")
display(missing_values[missing_values > 0].sort_values(ascending=False))
```

```
Missing values in each column:
```

	0
deck	688
age	177
embarked	2
embark_town	2

dtype: int64

5. Handle Missing Values

```
# Numerical columns: fill missing values with median
if "age" in df.columns:
    df["age"] = df["age"].fillna(df["age"].median())

# Categorical columns: fill missing values with mode
if "embarked" in df.columns:
    df["embarked"] = df["embarked"].fillna(df["embarked"].mode()[0])

if "embark_town" in df.columns:
    df["embark_town"] = df["embark_town"].fillna(df["embark_town"].mode()[0])

# Deck has many missing values, so it is removed
if "deck" in df.columns:
    df = df.drop(columns=["deck"])

print("Missing values after treatment:")
display(df.isnull().sum())
```

```
Missing values after treatment:
```

	0
survived	0
pclass	0
sex	0
age	0
sibsp	0
parch	0
fare	0
embarked	0
class	0
who	0
adult_male	0
embark_town	0
alive	0
alone	0

dtype: int64

6. Detect and Remove Duplicate Records

```
duplicate_count = df.duplicated().sum()

print("Number of duplicate records:", duplicate_count)

df = df.drop_duplicates().reset_index(drop=True)

print("Dataset shape after duplicate removal:", df.shape)
```

```
Number of duplicate records: 116
Dataset shape after duplicate removal: (775, 14)
```

7. Standardize Categorical Values

```
# Standardize text formatting for categorical columns
categorical_columns = df.select_dtypes(include=["object"]).columns

for col in categorical_columns:
    df[col] = df[col].astype(str).str.strip().str.lower()

print("Categorical values standardized.")
display(df[categorical_columns].head())
```

```
Categorical values standardized.
```

	sex	embarked	class	who	embark_town	alive
0	male	s	third	man	southampton	no
1	female	c	first	woman	cherbourg	yes
2	female	s	third	woman	southampton	yes
3	female	s	first	woman	southampton	yes
4	male	s	third	man	southampton	no

8. Encode Categorical Variables

```
# Label Encoding for the 'sex' column
label_encoder = LabelEncoder()
df["sex_encoded"] = label_encoder.fit_transform(df["sex"])

# One-Hot Encoding for the 'embarked' column
df = pd.get_dummies(
    df,
    columns=["embarked"],
    prefix="embarked",
    dtype=int
)

print("Categorical variables encoded successfully.")
display(df.head())
```

```
Categorical variables encoded successfully.
```

	survived	pclass	sex	age	sibsp	parch	fare	class	who	adult_male	embark_town	alive	alone	sex_encoded	embark
0	0	3	male	22.0	1	0	7.2500	third	man	True	southampton	no	False	1	
1	1	1	female	38.0	1	0	71.2833	first	woman	False	cherbourg	yes	False	0	
2	1	3	female	26.0	0	0	7.9250	third	woman	False	southampton	yes	True	0	
3	1	1	female	35.0	1	0	53.1000	first	woman	False	southampton	yes	False	0	
4	0	3	male	35.0	0	0	8.0500	third	man	True	southampton	no	True	1	

9. Detect Outliers using Box Plots and IQR

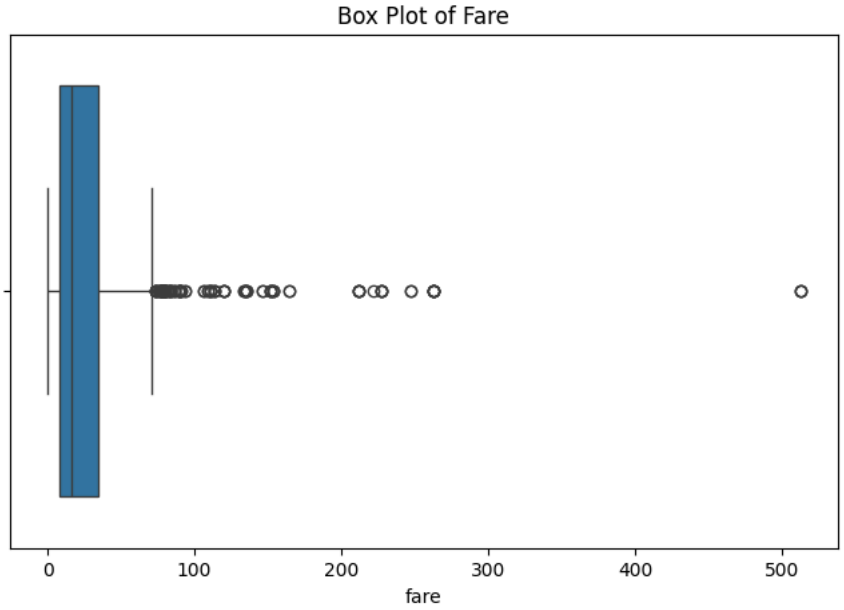
```
plt.figure(figsize=(8, 5))
sns.boxplot(x=df["fare"])
plt.title("Box Plot of Fare")
plt.show()

Q1 = df["fare"].quantile(0.25)
Q3 = df["fare"].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
```

```
outlier_mask = (df["fare"] < lower_bound) | (df["fare"] > upper_bound)

print("Number of Fare outliers:", outlier_mask.sum())
print("Lower bound:", lower_bound)
print("Upper bound:", upper_bound)
```




```
Number of Fare outliers: 102
Lower bound: -31.171850000000003
Upper bound: 73.41975000000001
```

10. Treat Outliers using IQR Capping

```
df["fare"] = df["fare"].clip(
    lower=lower_bound,
    upper=upper_bound
)

print("Outliers in Fare treated using IQR capping.")
```

```
Outliers in Fare treated using IQR capping.
```

11. Normalize Numerical Attributes

```
numeric_columns = ["age", "fare"]

scaler = MinMaxScaler()

df[numeric_columns] = scaler.fit_transform(df[numeric_columns])

print("Numerical attributes normalized using Min-Max scaling.")
display(df[numeric_columns].head())
```

```
Numerical attributes normalized using Min-Max scaling.
```

	age	fare
0	0.271174	0.098747
1	0.472229	0.970901
2	0.321438	0.107941
3	0.434531	0.723239
4	0.434531	0.109644

12. Feature Engineering

```
# Family Size = SibSp + Parch + 1
df["family_size"] = df["sibsp"] + df["parch"] + 1

# Create Age Group
df["age_group"] = pd.cut(
    df["age"],
    bins=[-0.01, 0.25, 0.60, 1.0],
    labels=["Young", "Adult", "Senior"]
)

print("Feature engineering completed.")
display(df[["age", "sibsp", "parch", "family_size", "age_group"]].head())
```

```
Feature engineering completed.
```

	age	sibsp	parch	family_size	age_group
0	0.271174	1	0	2	Adult
1	0.472229	1	0	2	Adult
2	0.321438	0	0	1	Adult
3	0.434531	1	0	2	Adult
4	0.434531	0	0	1	Adult

13. Final Dataset Inspection

```
print("Final dataset shape:", df.shape)
print("\nRemaining missing values:")
display(df.isnull().sum())

print("\nFinal dataset:")
display(df.head())
```

```
Final dataset shape: (775, 19)

Remaining missing values:
```

	0
survived	0
pclass	0
sex	0
age	0
sibsp	0
parch	0
fare	0
class	0
who	0
adult_male	0
embark_town	0
alive	0
alone	0
sex_encoded	0
embarked_c	0
embarked_q	0
embarked_s	0
family_size	0
age_group	0

dtype: int64

```
Final dataset:
```

	survived	pclass	sex	age	sibsp	parch	fare	class	who	adult_male	embark_town	alive	alone	sex_encoded	e
0	0	3	male	0.271174	1	0	0.098747	third	man	True	southampton	no	False	1	
1	1	1	female	0.472229	1	0	0.970901	first	woman	False	cherbourg	yes	False	0	
2	1	3	female	0.321438	0	0	0.107941	third	woman	False	southampton	yes	True	0	
3	1	1	female	0.434531	1	0	0.723239	first	woman	False	southampton	yes	False	0	
4	0	3	male	0.434531	0	0	0.109644	third	man	True	southampton	no	True	1	

14. Save Cleaned and Preprocessed Dataset

```
output_file = "Titanic_Cleaned_Preprocessed.csv"

df.to_csv(output_file, index=False)

print("Cleaned dataset saved as:", output_file)
```

```
Cleaned dataset saved as: Titanic_Cleaned_Preprocessed.csv
```

15. Download the Cleaned Dataset

```
from google.colab import files

files.download(output_file)
```

RESULT

The Titanic real-world dataset was collected from a public source and successfully cleaned and preprocessed. Missing values were handled, duplicate records were removed, categorical variables were encoded, outliers were detected and treated using the IQR method, numerical features were normalized, and new features were created. The final preprocessed dataset was saved as a CSV file.

Answers to Experiment Questions

Q1. Why is data preprocessing considered one of the most important phases in data analytics?

Data preprocessing improves the quality, consistency, and reliability of raw data. It handles missing values, duplicates, inconsistent formats, categorical values, and outliers. Good preprocessing helps produce more reliable analysis and machine learning results.

Q2. Explain different methods of handling missing values with suitable examples.

Missing values can be handled by deleting rows or columns when the missing proportion is high or the records are not useful. Numerical missing values can be replaced using mean or median. Categorical missing values can be replaced using mode. More advanced methods include interpolation and model-based imputation.

Q3. Differentiate between Label Encoding and One-Hot Encoding.

Label Encoding assigns an integer to each category. For example, Male can be represented by 1 and Female by 0. One-Hot Encoding creates separate binary columns for each category. One-Hot Encoding is generally useful when the categories do not have a natural order.

Q4. What are outliers? How can they affect analytical results?

Outliers are observations that are unusually far from the other observations. They can strongly affect mean, standard deviation, correlations, regression models, and other analytical results. Outliers should be investigated and treated according to the problem rather than removed blindly.

Q5. Explain the difference between normalization and standardization.

Normalization generally scales values to a fixed range such as 0 to 1. A common formula is (x - min) / (max - min). Standardization transforms values so that they have approximately mean 0 and standard deviation 1 using (x - mean) / standard deviation.

Q6. Why should duplicate records be removed before analysis?

Duplicate records can give some observations more weight than others and may produce incorrect counts, statistics, and model results. Removing duplicate records improves data consistency.

Q7. What is feature engineering? Give two practical examples.

Feature engineering means creating useful new attributes from existing data. Examples include creating Family Size from SibSp and Parch in the Titanic dataset, and creating Age Group from the Age attribute.